

Optimal 91-Digit BN254 Recoding

A faster ArgoMAC scalar-multiplication circuit

Kriterion research submission

5 September 2026

Abstract

The ArgoMAC baseline uses 92 digits in base $2 - \omega$. This report gives a 91-digit construction for BN254 G1. The construction checks four GLV representatives. An exact Eisenstein-norm proof shows that one representative terminates in 91 rounds. Lean 4.24 checks the proof without new axioms or compiler-trusted evaluation. The digit count is optimal for the fixed seven-symbol alphabet. The matching Rust code also garbles independent output rows with Rayon.

1 Contribution

Let r be the BN254 scalar modulus. Let ω be the nontrivial cube root used by the G1 endomorphism. Set $\beta = 2 - \omega$. The digit alphabet is

$$D = \{0, \pm 1, \pm \omega, \pm \omega^2\}.$$

The norm of β is seven.

The baseline uses 92 fixed positions. The new construction uses 91 fixed positions for every scalar. It keeps the same alphabet and input-label shape. It removes one output MAC row, nine digit adaptors, and 11,430 fixed-permutation applications.

2 Four-corner GLV recoding

Use the Eisenstein norm

$$Q(a, b) = a^2 - ab + b^2.$$

The relevant BN254 constants are

$$\begin{aligned} A &= 147946756881789319000765030803803410728, \\ d &= 9931322734385697763, \\ C &= A + d, \\ r &= A^2 + Ad + d^2. \end{aligned}$$

The two GLV kernel vectors are

$$v_1 = (-A, d), \quad v_2 = (-d, -C).$$

Arkworks divides the two negative quotient numerators toward zero. Thus, its GLV state is the floor representative z_0 used by the proof. The selector checks

$$z_0, \quad z_0 + v_1, \quad z_0 + v_2, \quad z_0 + v_1 + v_2.$$

Each shift is in the reconstruction kernel. Thus, every candidate represents the same scalar.

For quotient remainders $u, v \in [0, r)$, scaling maps these four states to the four corners of one Eisenstein cell. The two triangle lemmas show that one corner satisfies

$$3Q(a, b) \leq r.$$

This is the exact covering-radius step.

3 Exact 91-round bound

One recurrence round selects a unit digit and divides by β . For a state with $Q(a, b) \leq m$, the selected digit gives

$$Q(a - u_d, b - v_d) \leq m + 1 + \lfloor \sqrt{4m} \rfloor.$$

Exact division gives

$$Q(a - u_d, b - v_d) = 7Q(a', b').$$

Define the inverse safe bound

$$\begin{aligned} B(0) &= 2, \\ B(n+1) &= 7B(n) + 1 - \left\lfloor \sqrt{4(7B(n) + 1)} \right\rfloor. \end{aligned}$$

Lean proves the generic induction rule

$$Q(a, b) \leq B(n) \implies \text{after}(n+1, (a, b)) = (0, 0).$$

It also checks

$$\begin{aligned} B(90) &= 8072890822786049911192470628436961768360750887255955910449276353626847573707, \\ r &\leq 3B(90). \end{aligned}$$

The numeric proof uses ten-round kernel-checked checkpoints. It does not use `native_decide`.

The main Lean results are:

- `shiftedInitialCorrect`;
- `existsCandidateNormBound`;
- `fourCandidatesSignedI128`;
- `existsCorrectShiftedInitialTerminates91`;
- `shortInitialTerminates91`;
- `shortInitialCorrect`.

The axiom audit reports only `propext`, `Classical.choice`, and `Quot.sound`.

4 Optimality

Lean checks

$$7^{90} < r < 7^{91}.$$

A fixed 90-position string over seven symbols has at most 7^{90} values. It cannot represent all r scalars. Thus, 91 is optimal for this fixed alphabet.

5 Circuit cost

Item	92 digits	91 digits	Change
Output MAC rows	92	91	−1
Digit adaptors	833	824	−9
Bit adaptors	211,582	209,296	−2,286
Fixed-permutation applications	1,057,910	1,046,480	−11,430
Public permutation instances	12,495	12,360	−135

The Lean direct-table benchmark used 21 measured runs on each commit. The Apple M5 paired medians were 146 ms for the 92-digit shared-pass commit and 144 ms for the 91-digit commit. This is a 1.4 percent reduction. This small timing change is within short-run noise. The exact operation-count reduction is the primary result. The larger prior shared-pass change reduced the original 989 ms median to 160 ms. Do not multiply these separate ratios.

6 Rust row parallelism

The Rust implementation allocates one job for each output row. It allocates the three child nonce domains before Rayon starts. It samples one fresh 32-byte ChaCha12 root from the caller RNG. Row j uses stream

$$0x666d3265635f6d61 + j$$

from word position zero. Indexed collection keeps the row order. Debug builds use the same RNG and nonce schedule in serial order. Release builds use Rayon.

The root seed is erased after row construction. Jobs borrow output keys. They do not copy those secret keys into a second vector. Fixed Boolean arrays replace heap-based coefficient index sets. Tests compare one-worker and four-worker tables. A second test compares all 273 allocated child nonces with the serial schedule.

On the same Apple M5, the ARMv8 AES row benchmark used 100 Criterion samples. The 92-row serial baseline estimate was 15.934 ms. The 91-row serial implementation estimate was 15.488 ms. The 91-row, 10-worker estimate was 2.704 ms. The parallel result is 5.89 times faster than the serial baseline. The protocol-level `encode` benchmark used 10 samples. It fell from 81.309 ms to 68.460 ms, a 15.8 percent reduction. The ARMv8 backend gain is separate from the algorithmic and scheduling gains.

7 Security scope

The following invariants stay fixed:

- Every candidate reconstructs the same scalar modulo r .
- Every public circuit has exactly 91 output positions.
- The digit alphabet does not change.
- The input keeps 508 Lamport label positions.
- The fixed-key window loads stay 91, 91, and 72.
- Each row uses a separate nonce subtree and RNG stream.
- Worker order cannot change the table or caller RNG state.

The Rust selector uses four fixed 91-round tests and `subtle::Choice`. This gives a fixed iteration count. Lean proves that every tested coordinate fits a signed 128-bit integer. The report does not claim compiler-level constant time.

The digit theorem does not repair two baseline obligations. The current affine formula has an exceptional input that breaks perfect correctness. The Rust permutation schedule also lacks a complete adaptive-privacy reduction. The pinned challenge interface is not a closed Lean type and has no benchmark function. Therefore, this change is a proved performance contribution, but it is not a valid final challenge certificate at the pinned challenge revision.

8 Reproduction

The Lean proof uses Lean 4.24 and the pinned Mathlib checkout. Run:

```
lake build Proof Correctness
lake env lean entry/tests/Correctness.lean
lake exe bench
```

The Rust validation uses Rust 1.94.0. Run:

```
cargo fmt --all -- --check
cargo clippy --all-targets --all-features -- -D warnings
cargo test --all-features
cargo test --all-features --release
RAYON_NUM_THREADS=10 cargo bench --all-features \
  --bench field_mac_to_ec_mac -- 'field_mac_to_ec_mac/g1/garble'
```

References

- J. Bosma and H. W. Lenstra, “Complete systems of two addition laws for elliptic curves,” *Journal of Number Theory*, 1995.
- J. Renes, C. Costello, and L. Batina, “Complete addition formulas for prime order elliptic curves,” EURO-CRYPT 2016.
- RustCrypto, *AES 0.8.4 documentation*, ARMv8 run-time detection section.
- Arkworks, *ark-ec 0.5.0*, GLV scalar decomposition implementation.